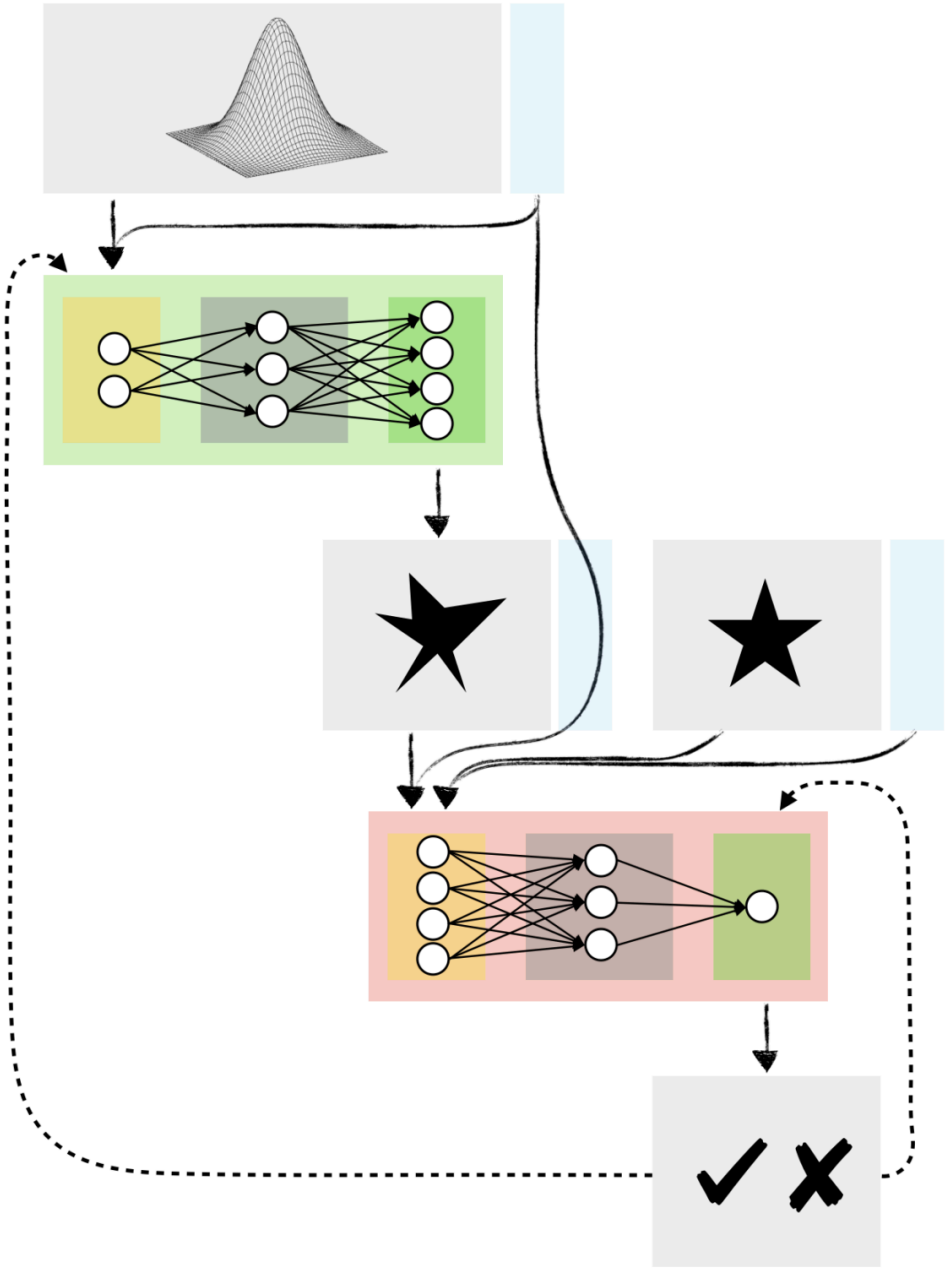
DATA SCIENCE

cGAN: Conditional Generative Adversarial Network – How to Gain Control Over GAN Outputs

An explanation of cGAN architecture with a detailed Python example

Saul Dobilas
Aug 1, 2022 • 13 min read

Neural Networks



Conditional Generative Adversarial Network. Image by author.

Intro

Have you experimented with Generative Adversarial Networks (GANs) yet? If so, you may have encountered a situation where you wanted your GAN to generate a **specific type of data** but did not have sufficient control over GANs outputs.

For example, assume you used a broad spectrum of flower images to train a GAN capable of producing fake pictures of flowers. While you can use your model to generate an image of a random flower, you cannot instruct it to create an image of, say, a tulip or a sunflower.

Conditional GAN (cGAN) allows us to condition the network with additional information such as class labels. It means that during the training, we pass images to the network with their actual labels (rose, tulip, sunflower etc.) for it to learn the difference between them. That way, we gain the ability to ask our model to generate images of specific flowers.

Contents

In this article, I will take you through the following:

- The place of Conditional GAN (cGAN) within the universe of Machine Learning algorithms
- An overview of cGAN and cDCGAN architecture and its components
- Python example showing you how to build a Conditional DCGAN from scratch with Keras / Tensorflow

Conditional GAN (cGAN) within the universe of Machine Learning algorithms

While most types of Neural Networks are Supervised, some, like Autoencoders, are Self-Supervised. Because of this and their unique approach to Machine Learning, I have given Neural Networks their own category in my ML Universe chart.

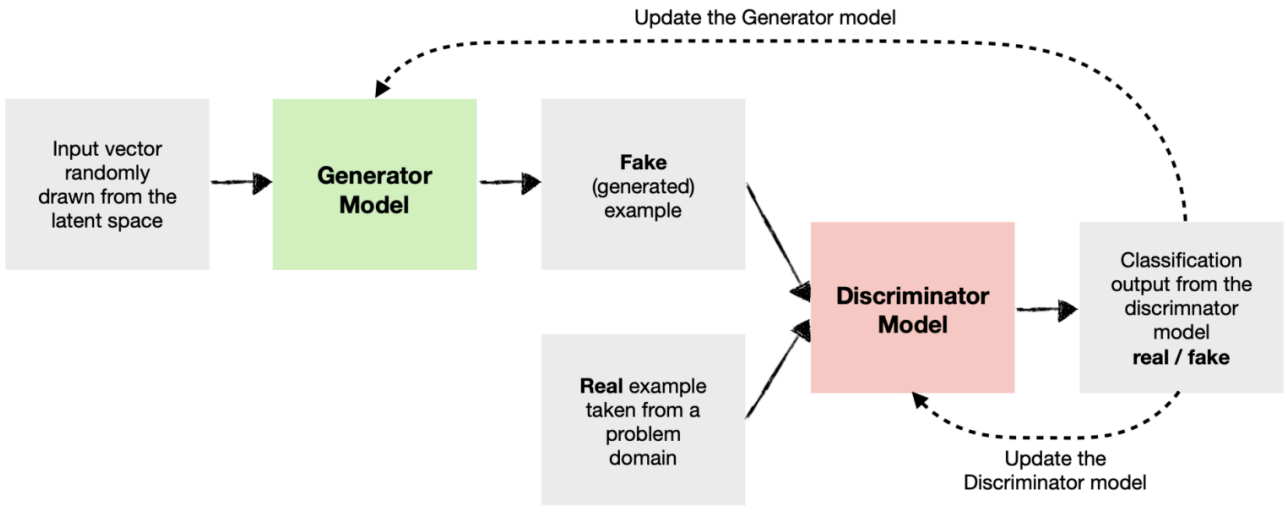
Since Conditional GAN is a type of GAN, you will find it under the Generative Adversarial Networks subcategory. Click 👇 on the **interactive chart** below to locate cGAN and to reveal other algorithms hiding under each branch of ML.

***If you enjoy Data Science and Machine Learning**, please subscribe to get an email with my new articles.*

An overview of cGAN architecture and its components

How do we condition a GAN?

Let's first remind ourselves of a basic Generative Adversarial Network architecture.



Basic GAN model architecture. Image by author.

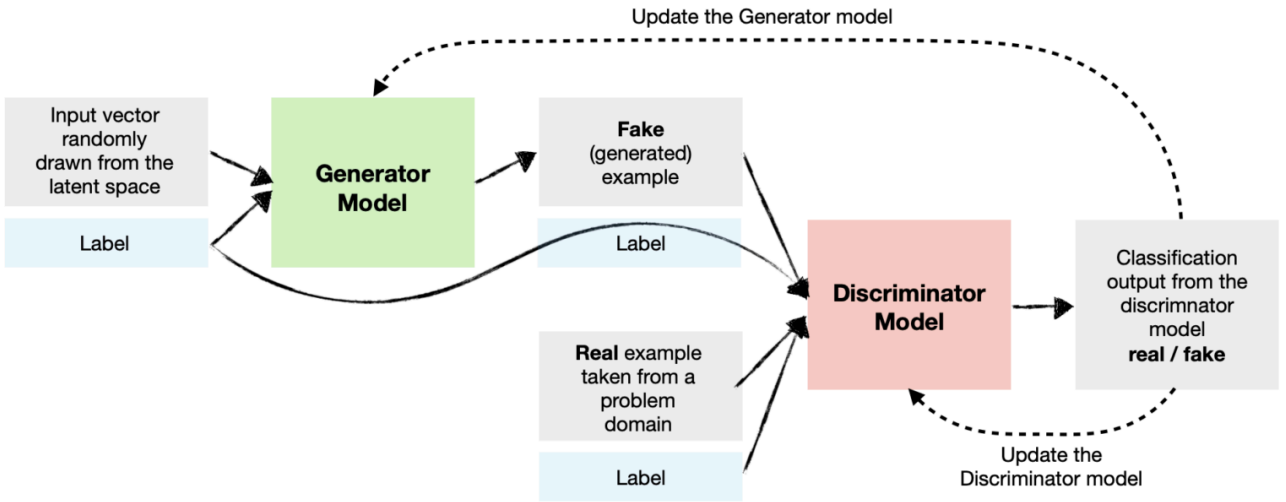
As you can see, we have two main components:

- **Generator Model** – generates new data (i.e., fake data) similar to that of the problem domain.
- **Discriminator Model** – tries to identify whether the provided example is fake (comes from a generator) or real (comes from the actual data domain).

In the case of a Conditional GAN, **we want to condition both** the Generator and the Discriminator so they know which type they are dealing with.

Say we use our GAN to create synthetic data containing house prices in London and Madrid. To make it conditional, we need to tell the Generator which city to generate the data for each time. We also need to inform the Discriminator whether the example passed to it is for London or Madrid.

So the Conditional GAN model architecture would look like this:



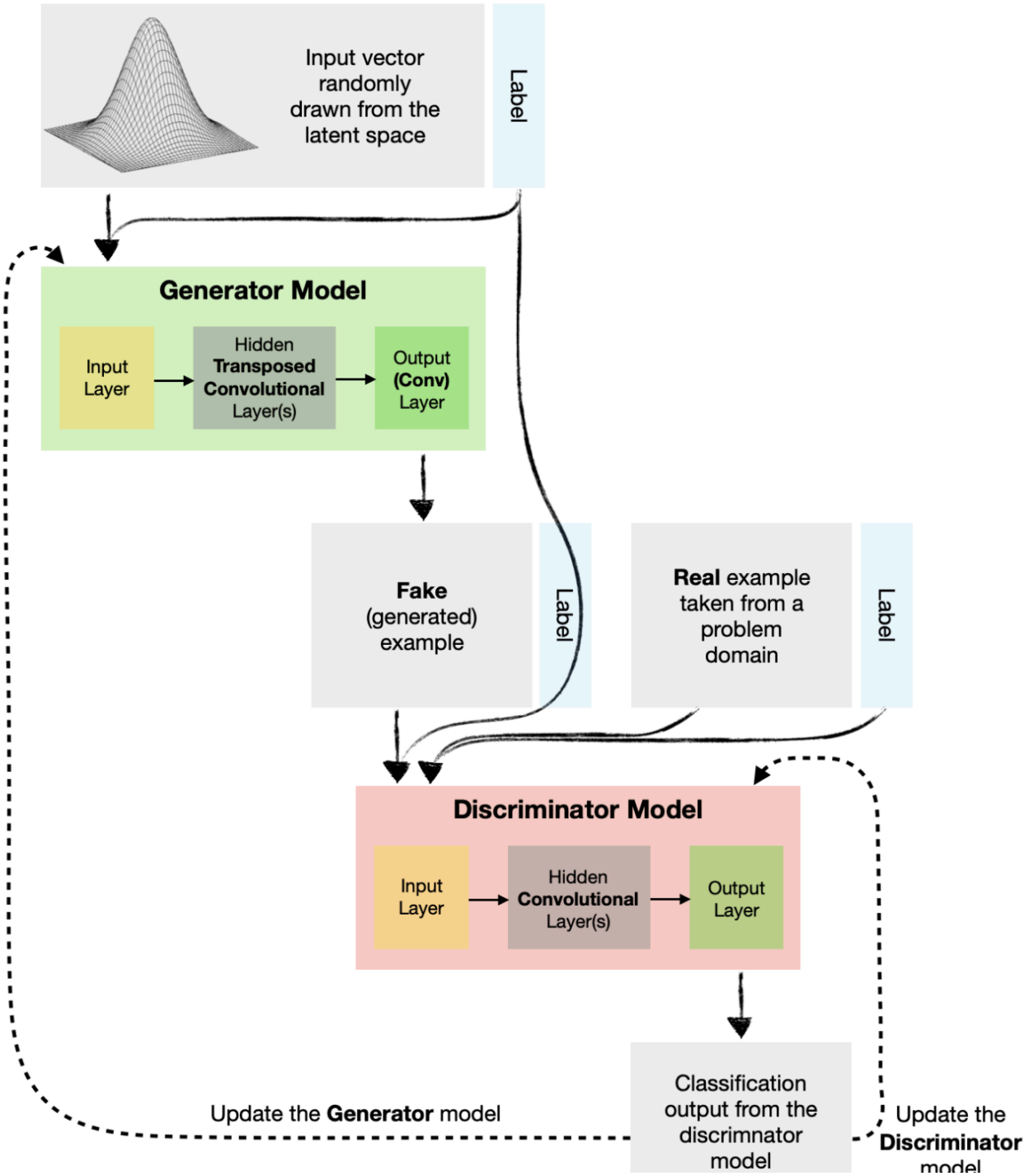
Conditional GAN (cGAN) model architecture. Image by author.

Note that we can condition GANs on many types of inputs. For example, we could also condition the network on other images where we want to create a GAN for image-to-image translation (e.g., turning the daytime image into a nighttime one).

Conditional Deep Convolutional GAN (cDCGAN)

As with the earlier flower example, we may want to condition a Deep Convolution GAN so we can ask the model to generate a specific type of image.

Below is a model architecture diagram for a Conditional DCGAN. Note that the high-level architecture is essentially the same as in the previous example, except the Generator and Discriminator contain additional layers, such as Convolutions and Transposed Convolutions.


Input vector randomly drawn from the latent space
Label
Generator Model
Input Layer
Hidden Transposed Convolutional Layer(s)
Output (Conv) Layer
Fake (generated) example
Label
Real example taken from a problem domain
Label
Discriminator Model
Input Layer
Hidden Convolutional Layer(s)
Output Layer
Classification output from the discrimnator model
Update the Generator model
Update the Discriminator model

real / fake

Conditional Deep Convolutional Generative Adversarial Network (cDCGAN). Image by author.

Python example

In this example, I will show you how to build a cDCGAN demonstrated in the above diagram. It will enable us to generate "fake" handwritten digits similar to those in the MNIST dataset.

Since we are building a conditional GAN, we will be able to specify which digit (0–9) we want the Generator to produce each time.

Setup

We will need to get the following data and libraries:

- MNIST handwritten digit data (copyright held by Yann LeCun and Corinna Cortes under the Creative Commons Attribution-Share Alike 3.0 license; the original source of the data: The MNIST Database)
- Numpy for data manipulation
- Matplotlib and Graphviz for some basic visualisations
- Tensorflow/Keras for Neural Networks

Let's import the libraries:

```
# Tensorflow / Keras
from tensorflow import keras # for building Neural Networks
print('Tensorflow/Keras: %s' % keras.__version__) # print version
from keras.models import Model, load_model # for assembling a Neur
from keras.layers import Input, Dense, Embedding, Reshape, Concate
from keras.layers import Conv2D, Conv2DTranspose, MaxPool2D, ReLU,
from tensorflow.keras.utils import plot_model # for plotting model
from tensorflow.keras.optimizers import Adam # for model optimizat

# Data manipulation
import numpy as np # for data manipulation
print('numpy: %s' % np.__version__) # print version
```

```
# Visualization
import matplotlib
import matplotlib.pyplot as plt # for data visualizationa
print('matplotlib: %s' % matplotlib.__version__) # print version
import graphviz # for showing model diagram
print('graphviz: %s' % graphviz.__version__) # print version

# Other utilities
import sys
import os

# Assign main directory to a variable
main_dir=os.path.dirname(sys.path[0])
#print(main_dir)
```

The above code prints package versions used in this example:

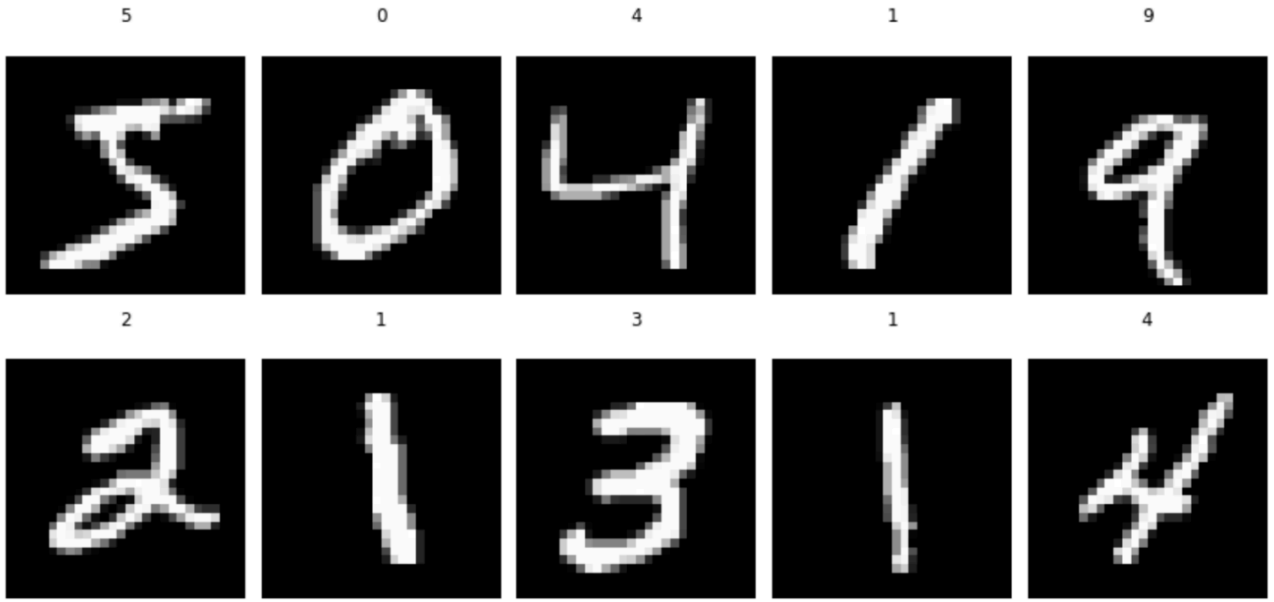
```
Tensorflow/Keras: 2.7.0
numpy: 1.21.4
matplotlib: 3.5.1
graphviz: 0.19.1
```

Next, we load the MNIST digit data, which is available in Keras datasets.

```
# Load digits data
(X_train, y_train), (_, _) = keras.datasets.mnist.load_data()

# Print shapes
print("Shape of X_train: ", X_train.shape)
print("Shape of y_train: ", y_train.shape)

# Display images of the first 10 digits in the training set and th
fig, axs = plt.subplots(2, 5, sharey=False, tight_layout=True, fig
n=0
for i in range(0,2):
    for j in range(0,5):
        axs[i,j].matshow(X_train[n], cmap='gray')
        axs[i,j].set(title=y_train[n])
        axs[i,j].axis('off')
        n=n+1
plt.show()

# Scale and reshape as required by the model
data=X_train.copy()
data=data.reshape(X_train.shape[0], 28, 28, 1)
data = (data - 127.5) / 127.5  # Normalize the images to [-1, 1]
print("Shape of the scaled array: ", data.shape)
```

The above code displays the first ten digits with their labels.

```
Shape of X_train:  (60000, 28, 28)
Shape of y_train:  (60000,)
```




```
Shape of the scaled array:  (60000, 28, 28, 1)
```

The first ten digits in the MNIST training data set. Image by author.

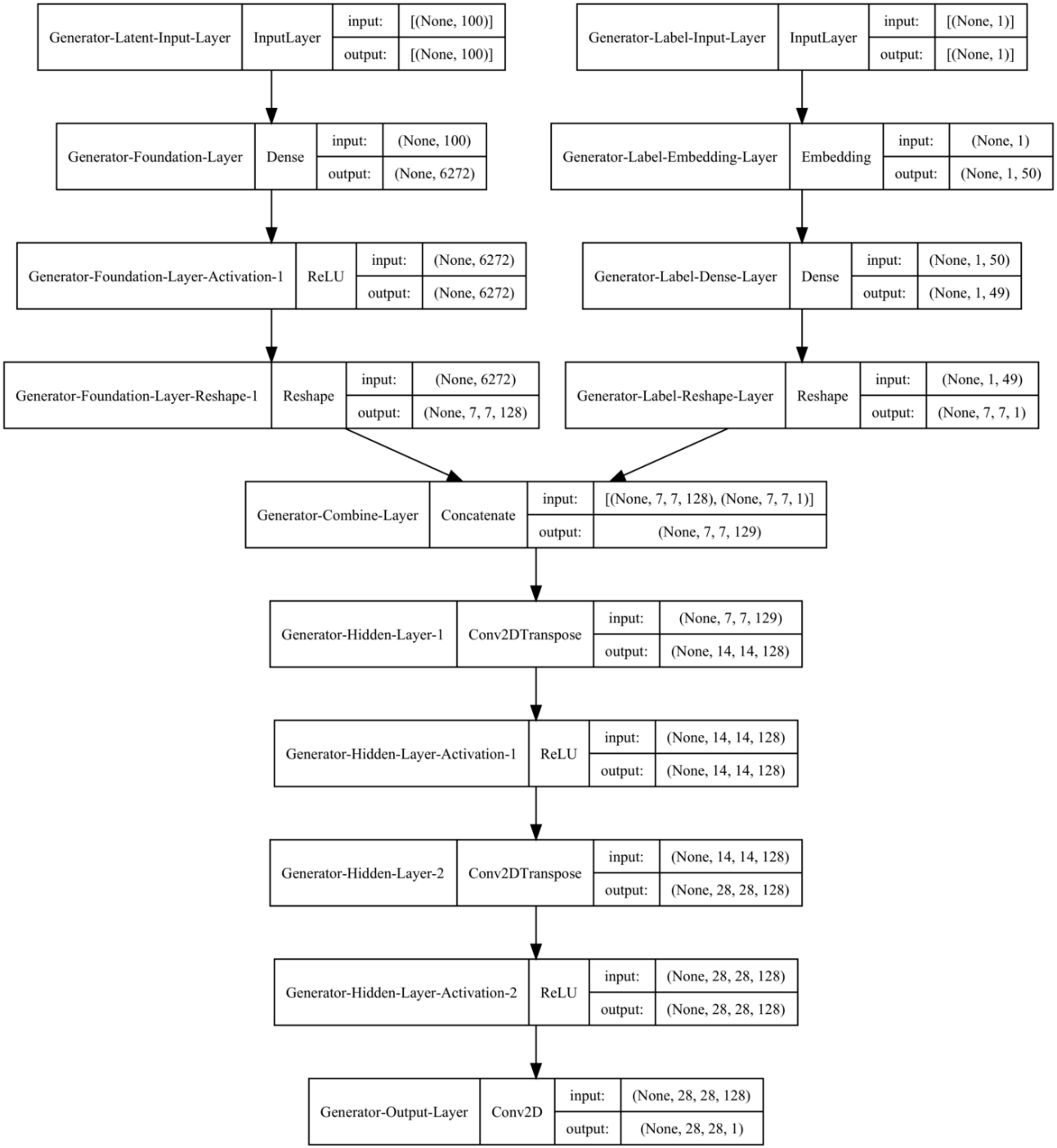
Creating a Conditional DCGAN model

With data preparation completed, let's define and assemble our models. Note that we will use **Keras Functional API**, which gives us more flexibility than the Sequential API, allowing us to create complex network architectures.

We will **start with the Generator:**

```
def generator(latent_dim, in_shape=(7,7,1), n_cats=10):

    # Label Inputs
    in_label = Input(shape=(1,), name='Generator-Label-Input-Layer
    lbls = Embedding(n_cats, 50, name='Generator-Label-Embedding-L

    # Scale up to image dimensions
    n_nodes = in_shape[0] * in_shape[1]
    lbls = Dense(n_nodes, name='Generator-Label-Dense-Layer')(lbls
    lbls = Reshape((in_shape[0], in_shape[1], 1), name='Generator-

    # Generator Inputs (latent vector)
    in_latent = Input(shape=latent_dim, name='Generator-Latent-Inp

    # Image Foundation
    n_nodes = 7 * 7 * 128 # number of nodes in the initial layer
    g = Dense(n_nodes, name='Generator-Foundation-Layer')(in_laten
    g = ReLU(name='Generator-Foundation-Layer-Activation-1')(g)
    g = Reshape((in_shape[0], in_shape[1], 128), name='Generator-F

    # Combine both inputs so it has two channels
    concat = Concatenate(name='Generator-Combine-Layer')([g, lbls]

    # Hidden Layer 1
    g = Conv2DTranspose(filters=128, kernel_size=(4,4), strides=(2
    g = ReLU(name='Generator-Hidden-Layer-Activation-1')(g)

    # Hidden Layer 2
    g = Conv2DTranspose(filters=128, kernel_size=(4,4), strides=(2
```

```
    g = ReLU(name='Generator-Hidden-Layer-Activation-2')(g)

    # Output Layer (Note, we use only one filter because we have a
    output_layer = Conv2D(filters=1, kernel_size=(7,7), activation

    # Define model
    model = Model([in_latent, in_label], output_layer, name='Gener
    return model

# Instantiate
latent_dim=100 # Our latent space has 100 dimensions. We can chang
gen_model = generator(latent_dim)

# Show model summary and plot model diagram
gen_model.summary()
plot_model(gen_model, show_shapes=True, show_layer_names=True, dpi
```


Generator-Latent-Input-Layer
InputLayer
input: [(None, 100)]
output: [(None, 100)]
Generator-Label-Input-Layer
InputLayer
input: [(None, 1)]
output: [(None, 1)]
Generator-Foundation-Layer
Dense
input: (None, 100)
output: (None, 6272)
Generator-Label-Embedding-Layer
Embedding
input: (None, 1)
output: (None, 1, 50)
Generator-Foundation-Layer-Activation-1
ReLU
input: (None, 6272)
output: (None, 6272)
Generator-Label-Dense-Layer
Dense
input: (None, 1, 50)
output: (None, 1, 49)
Generator-Foundation-Layer-Reshape-1
Reshape
input: (None, 6272)
output: (None, 7, 7, 128)
Generator-Label-Reshape-Layer
Reshape
input: (None, 1, 49)
output: (None, 7, 7, 1)
Generator-Combine-Layer
Concatenate
input: [(None, 7, 7, 128), (None, 7, 7, 1)]
output: (None, 7, 7, 129)
Generator-Hidden-Layer-1
Conv2DTranspose
input: (None, 7, 7, 129)
output: (None, 14, 14, 128)
Generator-Hidden-Layer-Activation-1
ReLU
input: (None, 14, 14, 128)
output: (None, 14, 14, 128)
Generator-Hidden-Layer-2
Conv2DTranspose
input: (None, 14, 14, 128)
output: (None, 28, 28, 128)
Generator-Hidden-Layer-Activation-2
ReLU
input: (None, 28, 28, 128)
output: (None, 28, 28, 128)
Generator-Output-Layer
Conv2D
input: (None, 28, 28, 128)
output: (None, 28, 28, 1)

Generator model diagram. Image by author.

We have **two inputs** to a Generator model. The first is a 100-node latent vector, which is a seed for our model, and the second is a label (0–9).

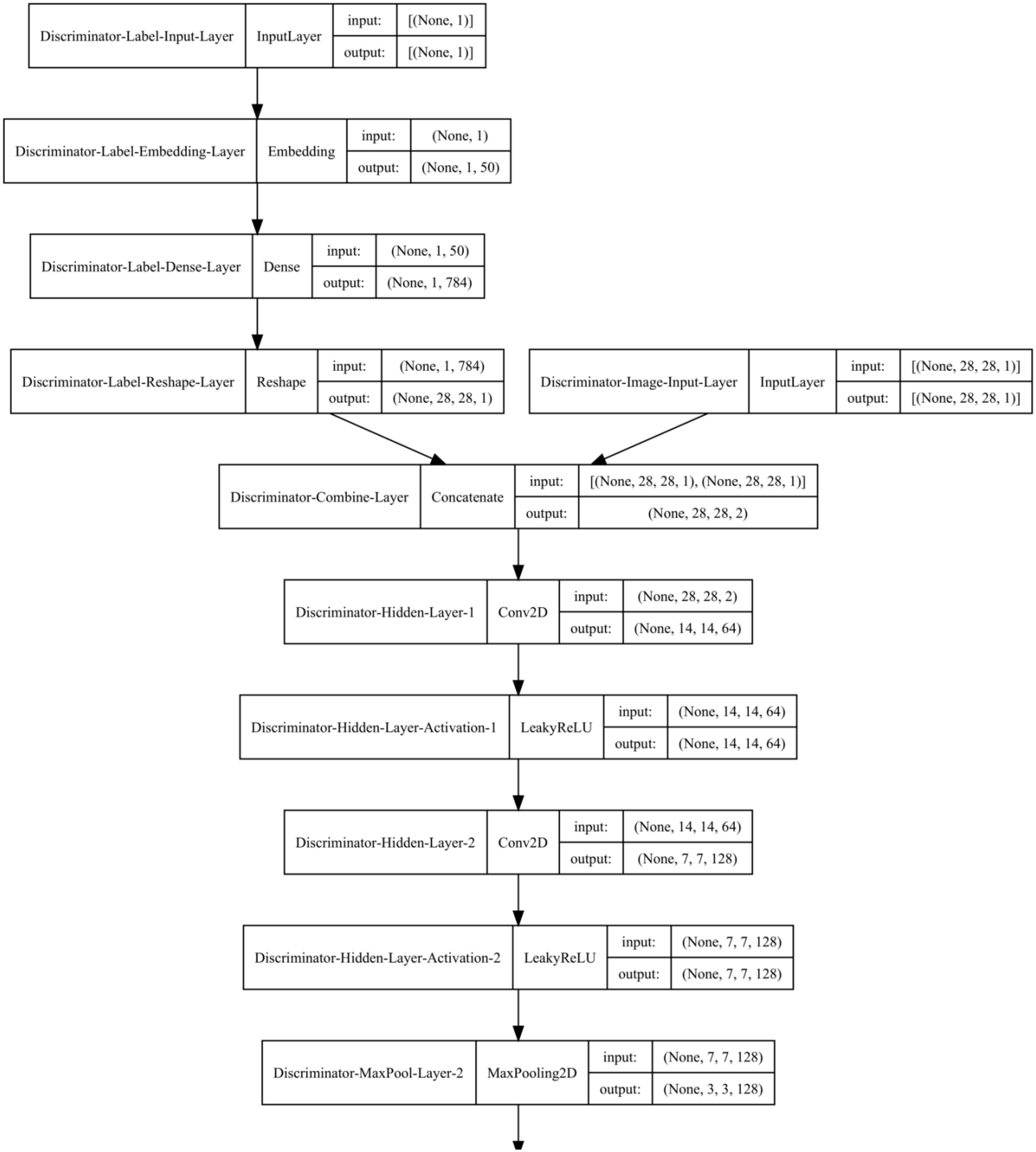
The latent vector and the label are reshaped and concatenated before they go through the rest of the network, where Transposed Convolutional layers upscale the data to the desired size (28 x 28 pixels).

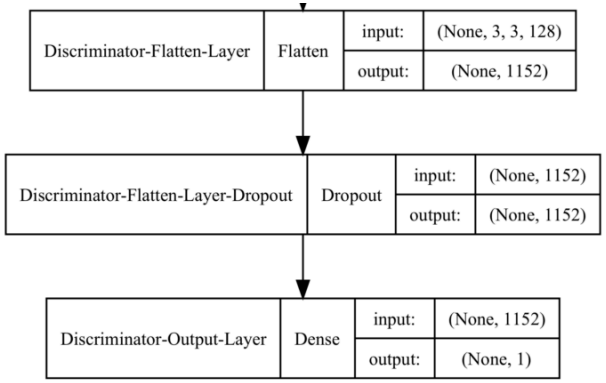
Next, let's define a **Discriminator model:**

```
def discriminator(in_shape=(28,28,1), n_cats=10):

    # Label Inputs
    in_label = Input(shape=(1,), name='Discriminator-Label-Input-L
    lbls = Embedding(n_cats, 50, name='Discriminator-Label-Embeddi

    # Scale up to image dimensions
    n_nodes = in_shape[0] * in_shape[1]
    lbls = Dense(n_nodes, name='Discriminator-Label-Dense-Layer')(
    lbls = Reshape((in_shape[0], in_shape[1], 1), name='Discrimina

    # Image Inputs
    in_image = Input(shape=in_shape, name='Discriminator-Image-Inp
```

```
    # Combine both inputs so it has two channels
    concat = Concatenate(name='Discriminator-Combine-Layer')([in_i

    # Hidden Layer 1
    h = Conv2D(filters=64, kernel_size=(3,3), strides=(2,2), paddi
    h = LeakyReLU(alpha=0.2, name='Discriminator-Hidden-Layer-Acti

    # Hidden Layer 2
    h = Conv2D(filters=128, kernel_size=(3,3), strides=(2,2), padd
    h = LeakyReLU(alpha=0.2, name='Discriminator-Hidden-Layer-Acti
    h = MaxPool2D(pool_size=(3,3), strides=(2,2), padding='valid',

    # Flatten and Output Layers
    h = Flatten(name='Discriminator-Flatten-Layer')(h) # Flatten t
    h = Dropout(0.2, name='Discriminator-Flatten-Layer-Dropout')(h

    output_layer = Dense(1, activation='sigmoid', name='Discrimina

    # Define model
    model = Model([in_image, in_label], output_layer, name='Discri

    # Compile the model
    model.compile(loss='binary_crossentropy', optimizer=Adam(learn
    return model

# Instantiate
dis_model = discriminator()
```

```
# Show model summary and plot model diagram
dis_model.summary()
plot_model(dis_model, show_shapes=True, show_layer_names=True, dpi
```


Discriminator-Label-Input-Layer
InputLayer
input: [(None, 1)]
output: [(None, 1)]
Discriminator-Label-Embedding-Layer
Embedding
input: (None, 1)
output: (None, 1, 50)
Discriminator-Label-Dense-Layer
Dense
input: (None, 1, 50)
output: (None, 1, 784)
Discriminator-Label-Reshape-Layer
Reshape
input: (None, 1, 784)
output: (None, 28, 28, 1)
Discriminator-Image-Input-Layer
InputLayer
input: [(None, 28, 28, 1)]
output: [(None, 28, 28, 1)]
Discriminator-Combine-Layer
Concatenate
input: [(None, 28, 28, 1), (None, 28, 28, 1)]
output: (None, 28, 28, 2)
Discriminator-Hidden-Layer-1
Conv2D
input: (None, 28, 28, 2)
output: (None, 14, 14, 64)
Discriminator-Hidden-Layer-Activation-1
LeakyReLU
input: (None, 14, 14, 64)
output: (None, 14, 14, 64)
Discriminator-Hidden-Layer-2
Conv2D
input: (None, 14, 14, 64)
output: (None, 7, 7, 128)
Discriminator-Hidden-Layer-Activation-2
LeakyReLU
input: (None, 7, 7, 128)
output: (None, 7, 7, 128)
Discriminator-MaxPool-Layer-2
MaxPooling2D
input: (None, 7, 7, 128)
output: (None, 3, 3, 128)



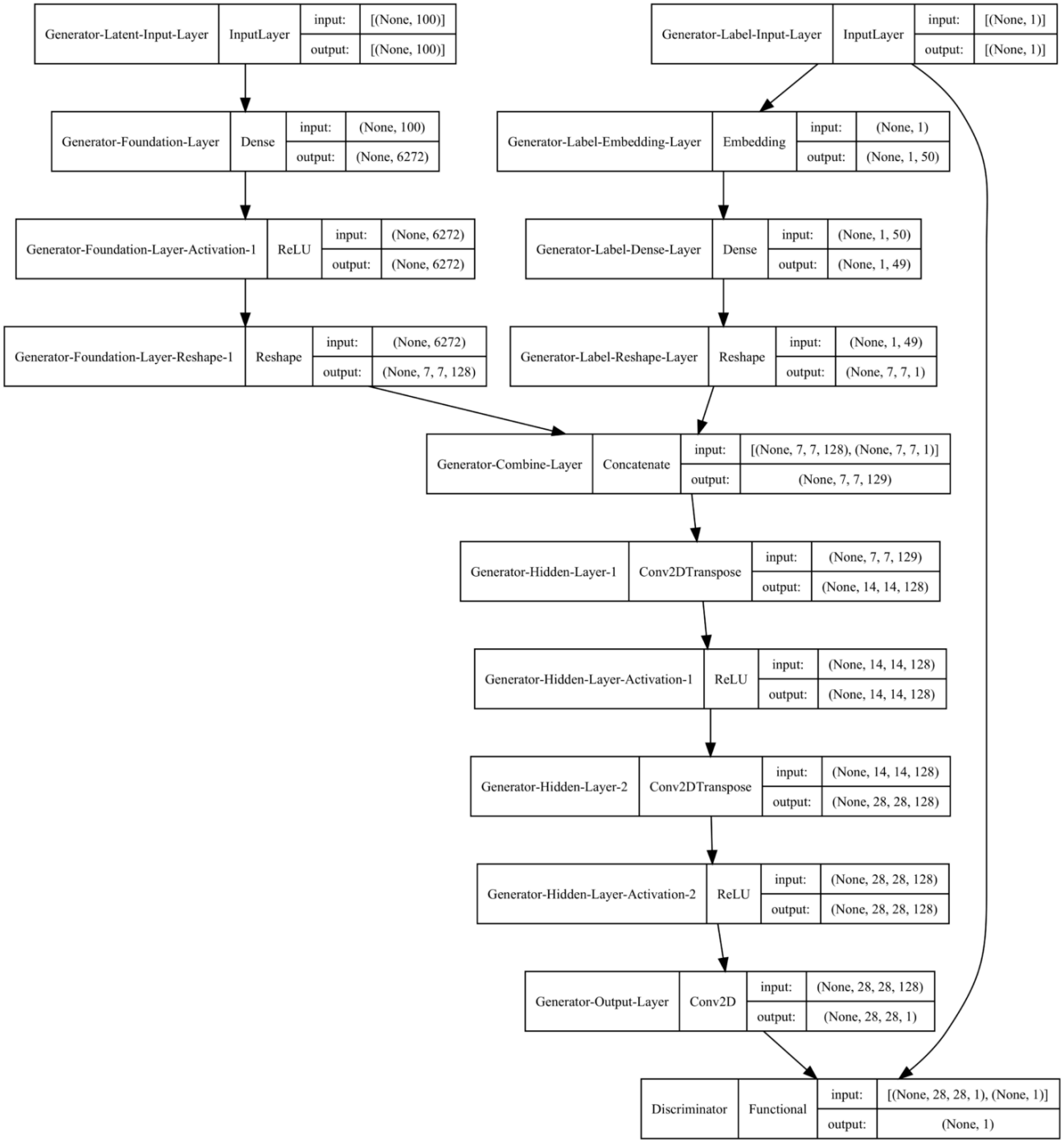
Discriminator model diagram. Image by author.

The Discriminator also has two separate inputs. One is a label, while the other is an image – either a real one from the MNSIT dataset or a fake one created by the Generator model.

The inputs are combined and passed through the network. The Convolutional and MaxPooling layers extract features and reduce the size before the prediction (real/fake) is made in the output layer.

Let's **combine the Generator and the Discriminator** to create a Conditional Deep Convolutional Adversarial Network (cDCGAN). One crucial detail in the code below is that we **make the Discriminator model non-trainable**. We do this because we want to train the Discriminator separately using a combination of real and fake (generated) data. You will see how we do that later.

```
def def_gan(generator, discriminator):

    # We don't want to train the weights of discriminator at this
    discriminator.trainable = False

    # Get Generator inputs / outputs
    gen_latent, gen_label = generator.input # Latent and label inp
    gen_output = generator.output # Generator output image

    # Connect image and label from the generator to use as input i
    gan_output = discriminator([gen_output, gen_label])

    # Define GAN model
    model = Model([gen_latent, gen_label], gan_output, name="cDCGA

    # Compile the model
    model.compile(loss='binary_crossentropy', optimizer=Adam(learn
    return model

# Instantiate
gan_model = def_gan(gen_model, dis_model)

# Show model summary and plot model diagram
gan_model.summary()
plot_model(gan_model, show_shapes=True, show_layer_names=True, dpi
```


Generator-Latent-Input-Layer
InputLayer
input: [(None, 100)]
output: [(None, 100)]
Generator-Label-Input-Layer
InputLayer
input: [(None, 1)]
output: [(None, 1)]
Generator-Foundation-Layer
Dense
input: (None, 100)
output: (None, 6272)
Generator-Label-Embedding-Layer
Embedding
input: (None, 1)
output: (None, 1, 50)
Generator-Foundation-Layer-Activation-1
ReLU
input: (None, 6272)
output: (None, 6272)
Generator-Label-Dense-Layer
Dense
input: (None, 1, 50)
output: (None, 1, 49)
Generator-Foundation-Layer-Reshape-1
Reshape
input: (None, 6272)
output: (None, 7, 7, 128)
Generator-Label-Reshape-Layer
Reshape
input: (None, 1, 49)
output: (None, 7, 7, 1)
Generator-Combine-Layer
Concatenate
input: [(None, 7, 7, 128), (None, 7, 7, 1)]
output: (None, 7, 7, 129)
Generator-Hidden-Layer-1
Conv2DTranspose
input: (None, 7, 7, 129)
output: (None, 14, 14, 128)
Generator-Hidden-Layer-Activation-1
ReLU
input: (None, 14, 14, 128)
output: (None, 14, 14, 128)
Generator-Hidden-Layer-2
Conv2DTranspose
input: (None, 14, 14, 128)
output: (None, 28, 28, 128)
Generator-Hidden-Layer-Activation-2
ReLU
input: (None, 28, 28, 128)
output: (None, 28, 28, 128)
Generator-Output-Layer
Conv2D
input: (None, 28, 28, 128)
output: (None, 28, 28, 1)
Discriminator
Functional
input: [(None, 28, 28, 1), (None, 1)]
output: (None, 1)

cDCGAN model diagram. Image by author.

Note how the Generator and the Discriminator use the same label as input.

Preparing inputs for the Generator and the Discriminator

We will create three simple functions that will aid us in sampling / generating data for the two models.

- The first function samples real images and labels from the training data;
- The second function draws random vectors from the latent space, as well as random labels to be used as inputs into the Generator;
- Finally, the third function passes latent variables and labels into the Generator model to generate fake examples.

```
def real_samples(dataset, categories, n):

    # Create a random list of indices
    indx = np.random.randint(0, dataset.shape[0], n)

    # Select real data samples (images and category labels) using
```

```
    X, cat_labels = dataset[indx], categories[indx]

    # Class labels
    y = np.ones((n, 1))
    return [X, cat_labels], y

def latent_vector(latent_dim, n, n_cats=10):

    # Generate points in the latent space
    latent_input = np.random.randn(latent_dim * n)

    # Reshape into a batch of inputs for the network
    latent_input = latent_input.reshape(n, latent_dim)

    # Generate category labels
    cat_labels = np.random.randint(0, n_cats, n)
    return [latent_input, cat_labels]

def fake_samples(generator, latent_dim, n):

    # Draw latent variables
    latent_output, cat_labels = latent_vector(latent_dim, n)

    # Predict outputs (i.e., generate fake samples)
    X = generator.predict([latent_output, cat_labels])

    # Create class labels
```

```
    y = np.zeros((n, 1))
    return [X, cat_labels], y
```

Model training and evaluation

The final two functions will help us train the models and display interim results (at specified intervals), so we can observe how the Generator improves over time.

Let's create a function to display interim results first:

```
def show_fakes(generator, latent_dim, n=10):

    # Get fake (generated) samples
    x_fake, y_fake = fake_samples(generator, latent_dim, n)

    # Rescale from [-1, 1] to [0, 1]
    X_tst = (x_fake[0] + 1) / 2.0

    # Display fake (generated) images
    fig, axs = plt.subplots(2, 5, sharey=False, tight_layout=True,
    k=0
    for i in range(0,2):
        for j in range(0,5):
            axs[i,j].matshow(X_tst[k], cmap='gray')
```

```
            axs[i,j].set(title=x_fake[1][k])
            axs[i,j].axis('off')
            k=k+1
    plt.show()
```

Finally, let's define the training function:

```
def train(g_model, d_model, gan_model, dataset, categories, latent
    # Number of batches to use per each epoch
    batch_per_epoch = int(dataset.shape[0] / n_batch)
    print(' batch_per_epoch: ',  batch_per_epoch)
    # Our batch to train the discriminator will consist of half re
    half_batch = int(n_batch / 2)

    # We will manually enumare epochs
    for i in range(n_epochs):

        # Enumerate batches over the training set
        for j in range(batch_per_epoch):

        # Discriminator training
            # Prep real samples
            [x_real, cat_labels_real], y_real = real_samples(datas
            # Train discriminator with real samples
            discriminator_loss1, _ = d_model.train_on_batch([x_rea

            # Prep fake (generated) samples
```

```
            [x_fake, cat_labels_fake], y_fake = fake_samples(g_mod
            # Train discriminator with fake samples
            discriminator_loss2, _ = d_model.train_on_batch([x_fak

        # Generator training
            # Get values from the latent space to be used as input
            [latent_input, cat_labels] = latent_vector(latent_dim,
            # While we are generating fake samples,
            # we want GAN generator model to create examples that
            # hence we want to pass labels corresponding to real s
            y_gan = np.ones((n_batch, 1))

            # Train the generator via a composite GAN model
            generator_loss = gan_model.train_on_batch([latent_inpu

        # Summarize training progress and loss
            if (j) % n_eval == 0:
                print('Epoch: %d, Batch: %d/%d, D_Loss_Real=%.3f,
                      (i+1, j+1, batch_per_epoch, discriminator_lo
                show_fakes(g_model, latent_dim)
```

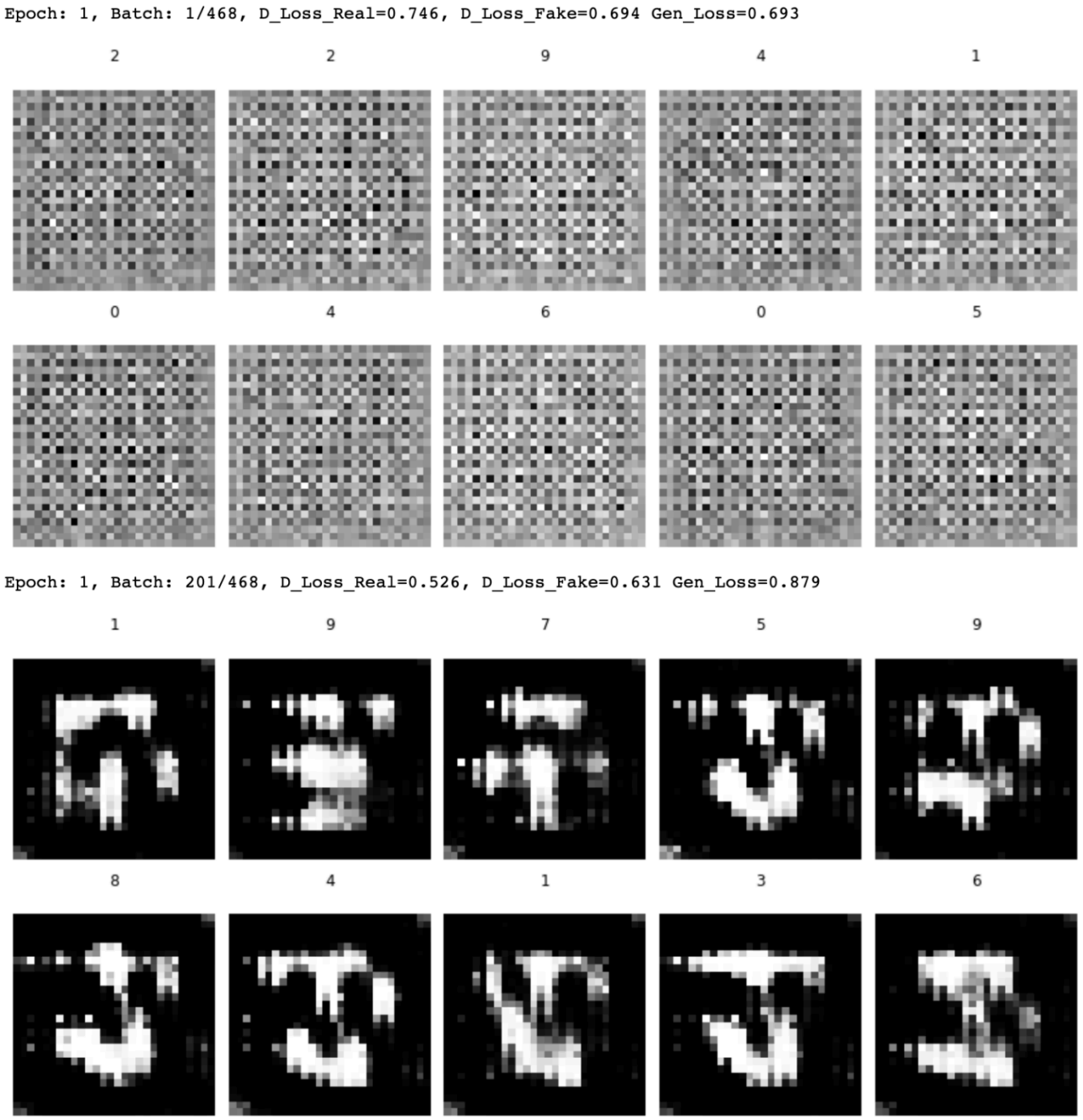
Now we can call our training function, get some tea and let the computer do the rest 😃

```
train(gen_model, dis_model, gan_model, data, y_train, latent_dim)
```

Results

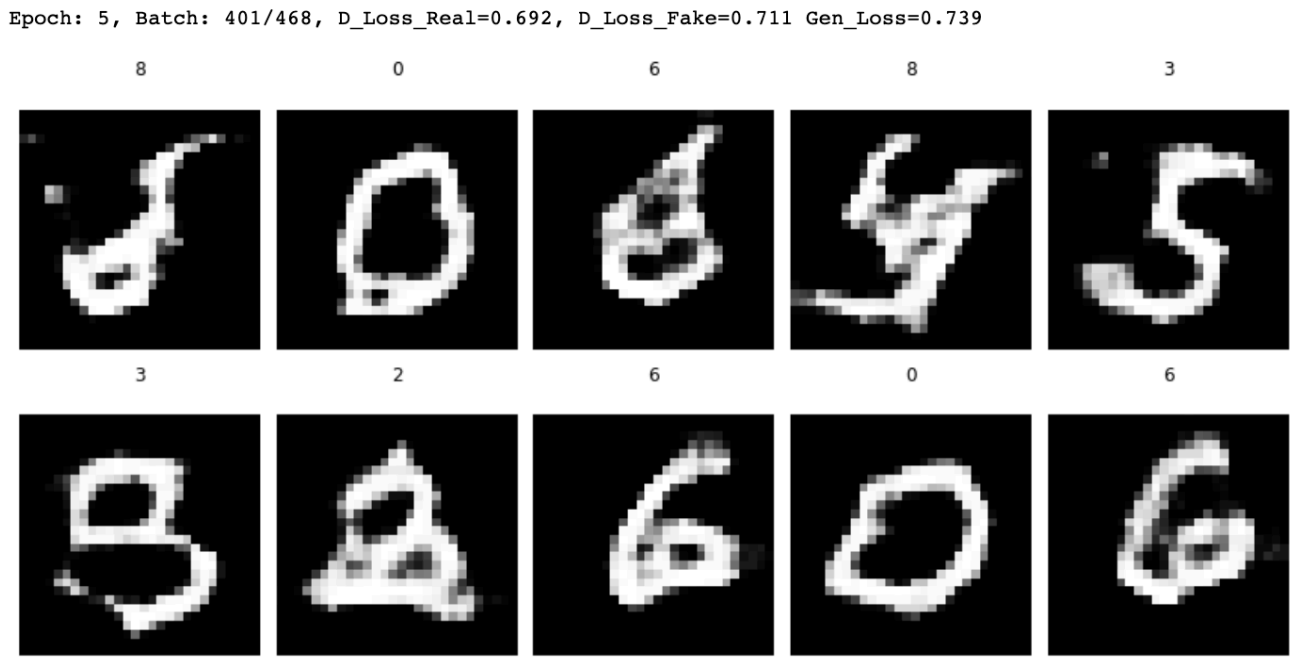
As the Generator and the Discriminator compete to outsmart each other, we can track their progress.

Here are some early attempts by the Generator to create handwritten digits:



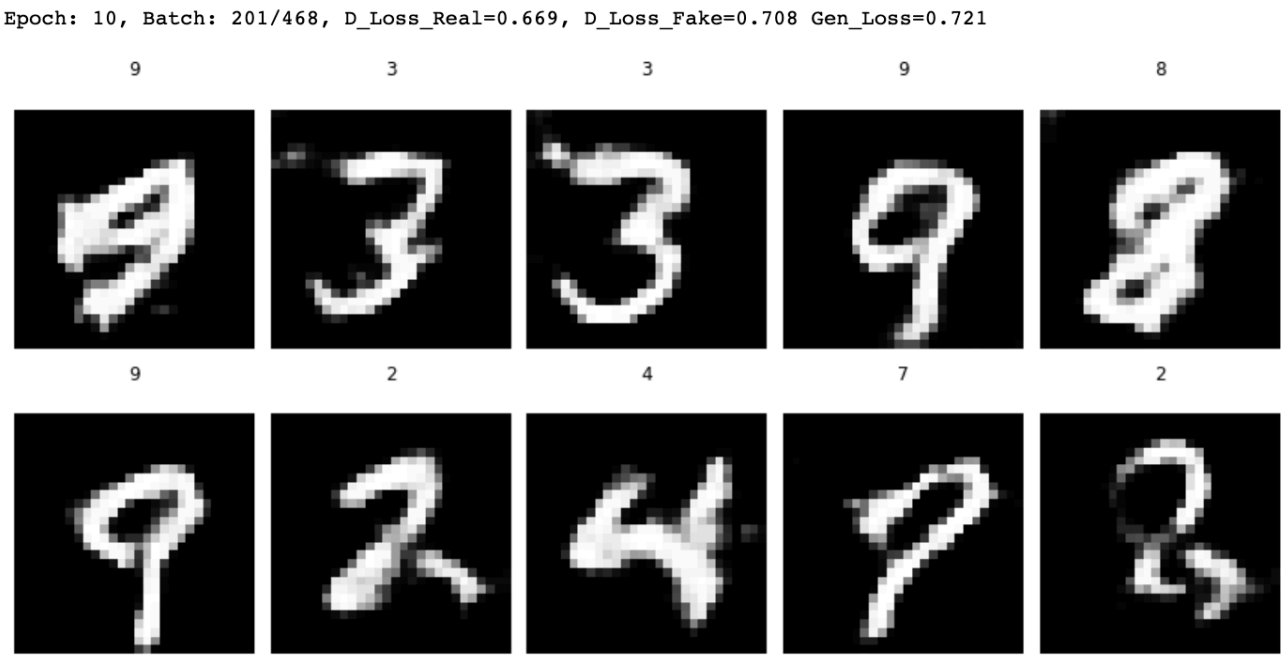
Early attempts by the Generator to create handwritten digits. Image by author.

Some progress by Epoch 5:



Improvement in the Generator results by Epoch 5. Image by author.

The Generator continues to get better throughout training with fake images by Epoch 10 looking like this:



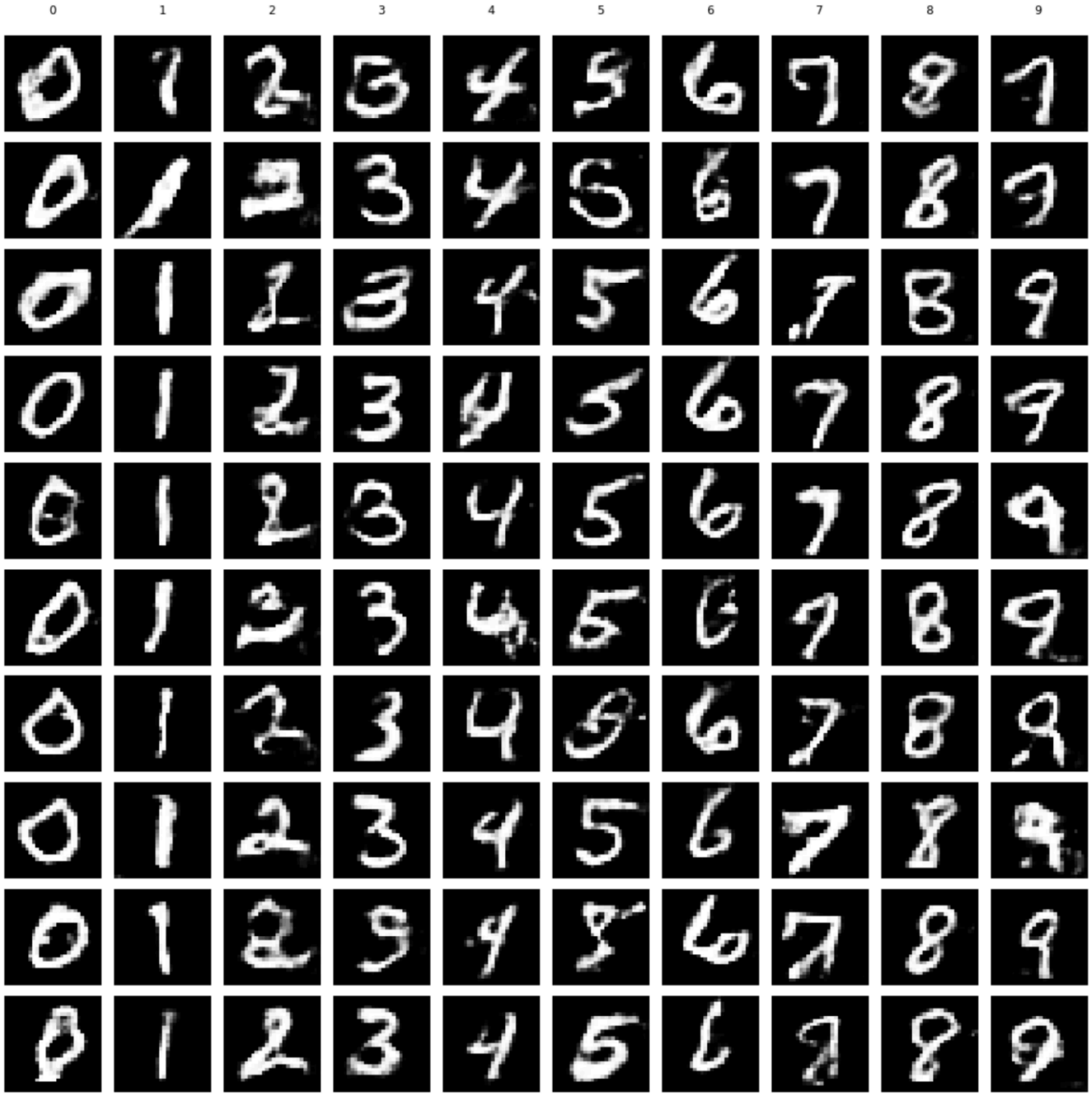
Improvement in the Generator results by Epoch 10. Image by author.

Once the model training is complete, we can save the Generator part for future use.

```
# We need to compile the generator to avoid a warning. This is bec
gen_model.compile(loss='binary_crossentropy', optimizer=Adam(learn
# Save the Generator on your drive
gen_model.save(main_dir+'/data/cgan_generator.h5')
```

Here is an example of how we can load the model and get it to generate images with specific labels:

```
# Generate latent points
latent_points, _ = latent_vector(100, 100)

# Specify labels that we want (0-9 repeated 10 times)
labels = np.asarray([x for _ in range(10) for x in range(10)])

# Load previously saved generator model
model = load_model(main_dir+'/data/cgan_generator.h5')

# Generate images
gen_imgs  = model.predict([latent_points, labels])

# Scale from [-1, 1] to [0, 1]
gen_imgs = (gen_imgs + 1) / 2.0

# Display images
fig, axs = plt.subplots(10, 10, sharey=False, tight_layout=True, f
k=0
for i in range(0,10):
    for j in range(0,10):
        axs[i,j].matshow(gen_imgs[k], cmap='gray')
        axs[0,j].set(title=labels[k])
        axs[i,j].axis('off')
        k=k+1
plt.show()
```



"Fake" handwritten digits generated with our cDCGAN model. Image by author.

As you can see, the results are not perfect, but we can improve them further by training the model for longer.

Final remarks

I hope my explanation and examples were sufficiently clear. Either way, please do not hesitate to leave a comment if you have any questions or suggestions. The complete Jupyter Notebook with the above Python code can be found on my **GitHub repository**.

Also, please feel free to check out my other GAN and Neural Network articles on Medium/TDS:

- Generative Adversarial Networks: GAN, DCGAN
- Convolutional Networks: DCN, Transposed CNN
- Recurrent Networks: RNN, LSTM, GRU
- Autoencoders: AE, DAE, VAE, SAE

If you would like to receive my upcoming articles on Machine Learning and Neural Networks, please subscribe with your email, and they will land in your inbox as soon as I publish them.

Cheers! 🤓 S**aul Dobilas**

WRITTEN BY

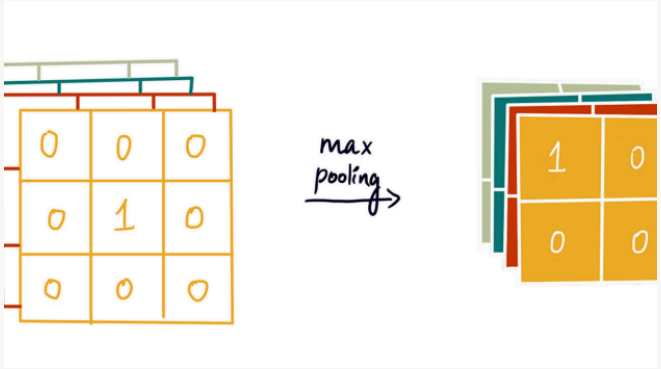
Saul Dobilas

See all from Saul Dobilas

Data Science | Machine Learning | Neural Networks | Python | Technology

Share This Article

Related Articles



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 • 6 min read

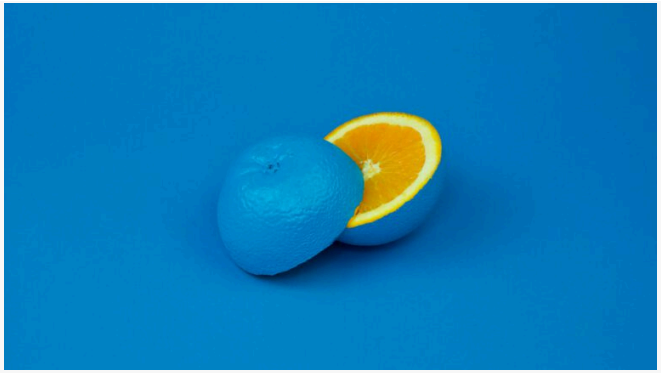
ARTIFICIAL INTELLIGENCE

How to Forecast Hierarchical Time Series

A beginner's guide to forecast reconciliation

Dr. Robert Kübler

August 20, 2024 • 13 min read



DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of...

Piero Paialunga

August 21, 2024 • 12 min read

MACHINE LEARNING

3 AI Use Cases (That Are Not a Chatbot)

Feature engineering, structuring unstructured data, and lead scoring

Shaw Talebi

August 21, 2024 • 7 min read

DATA SCIENCE

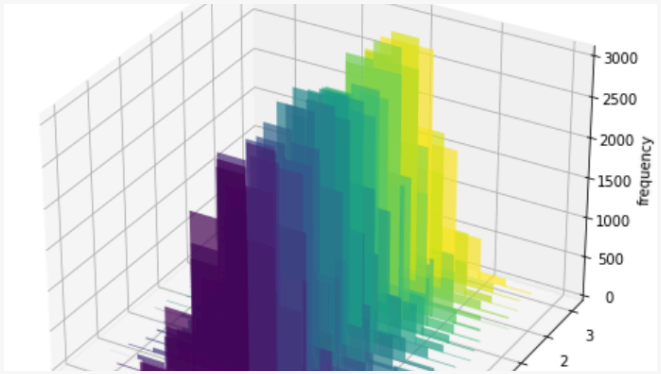
Solving a Constrained Project Scheduling Problem with Quantum Annealing

Solving the resource constrained project scheduling problem (RCPSP) with D-Wave's hybrid constrained quadratic model (CQM)

Luis Fernando PÉREZ ARMAS, Ph.D.

August 20, 2024 • 29 min read

DATA SCIENCE



DATA SCIENCE

Must-Know in Statistics: The Bivariate Normal Projection Explained

Derivation and practical examples of this powerful concept

Luigi Battistoni

August 14, 2024 • 7 min read

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 • 6 min read

towards
data science

Your home for data science and Al. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

Subscribe to Our Newsletter